

TEAM18

An OCR-Based Translation Pipeline GDGoC AI Challenges

Nguyễn Phan Hoàng Bách
Nguyễn Phúc Khang

Contents

For more details,
read the report in pdf



Training and Model usage

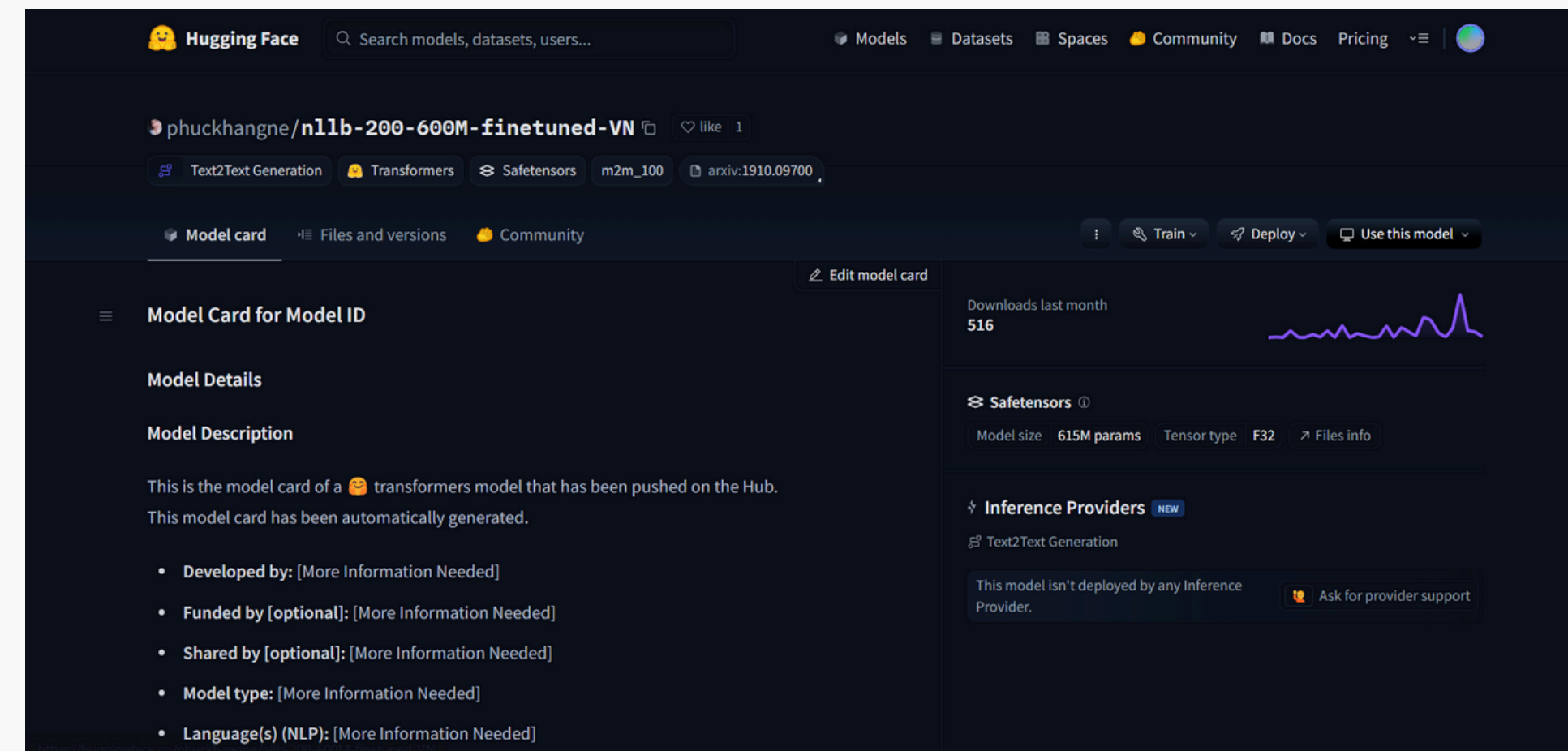


Building Application

Training and Models usage

Translation Model - Finetuning NLLB 200 600M

- We selected and finetuned NLLB200 for text translation within our system.
- NLLB200 is famous for its meticulous training by Meta, and its inherent multilingual capabilities.

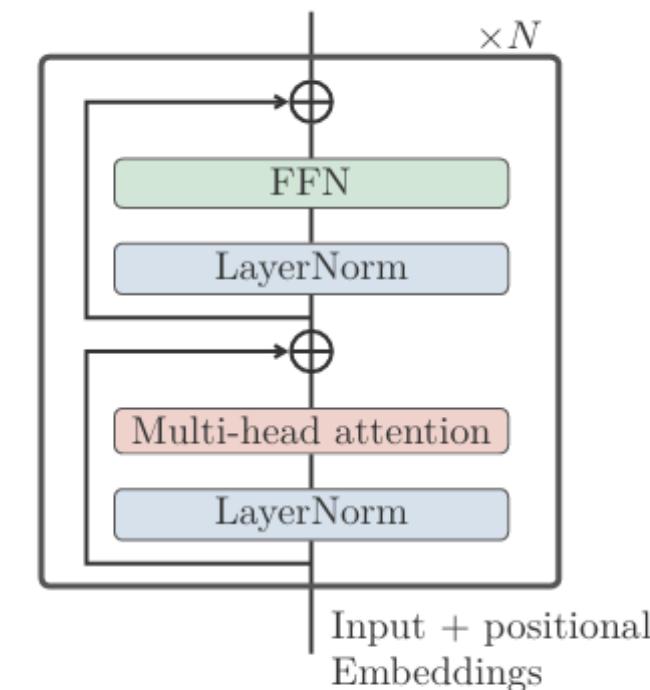


Training and Models usage

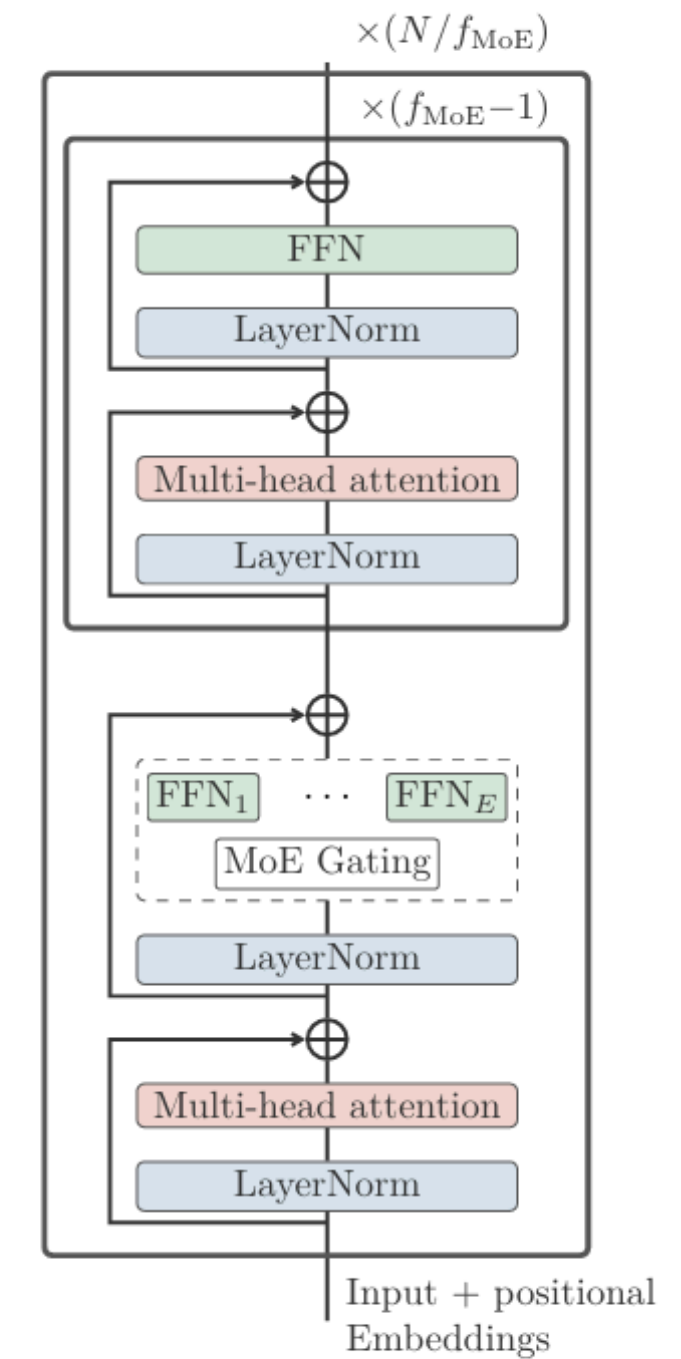
Translation Model - Finetuning NLLB 200 600M

The NLLB-200 architecture features:

- Shared Transformer Encoder/Decoder for cross-lingual learning,
- Language Tokens to guide target-language translation,
- Mixture-of-Experts (MoE) Layers to boost model capacity efficiently.
- Multilingual Training using back-translation, knowledge distillation, and temperature-based sampling to balance languages.



(a) Dense Transformer



(b) MoE Transformer

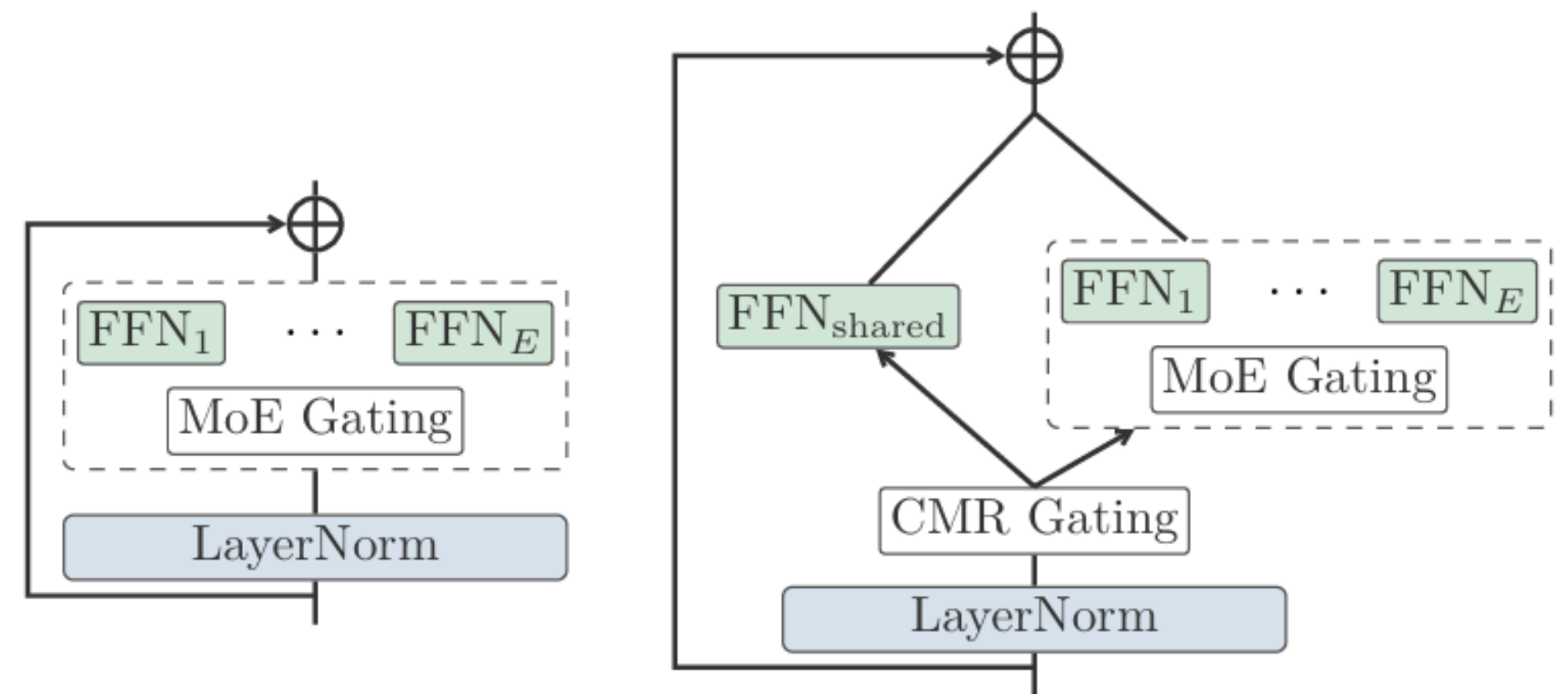
Model Architecture

Training and Models usage

Translation Model - Finetuning NLLB 200 600M

The NLLB-200 architecture features:

- Shared Transformer Encoder/Decoder for cross-lingual learning,
- Language Tokens to guide target-language translation,
- Mixture-of-Experts (MoE) Layers to boost model capacity efficiently.
- Multilingual Training using back-translation, knowledge distillation, and temperature-based sampling to balance languages.



Model Architecture

Training and Models usage

Translation Model - Finetuning NLLB 200 600M

- After thorough data cleaning, we **fine-tuned the NLLB-200** model using the Hugging Face Transformers library.
- The process involved two main phases: preparing tokenized data with explicit `src_lang` specification and **configuring training using `TrainingArguments` and `Seq2SeqTrainer`**, supported by `DataCollatorForSeq2Seq` for dynamic padding.
- The model achieved a **BLEU score of 0.68**, reflecting near human-level translation quality in our domain.

Training and Models usage

OCR Model - PaddleOCR

- PaddleOCR is an open-source OCR system by Baidu's PaddlePaddle team, built as a modular pipeline for detecting and recognizing text in multilingual and complex image layouts.
- We selected PaddleOCR as the baseline model for the OCR task due to its **robust support for multiple languages and comprehensive documentation**, which facilitates customization and finetuning

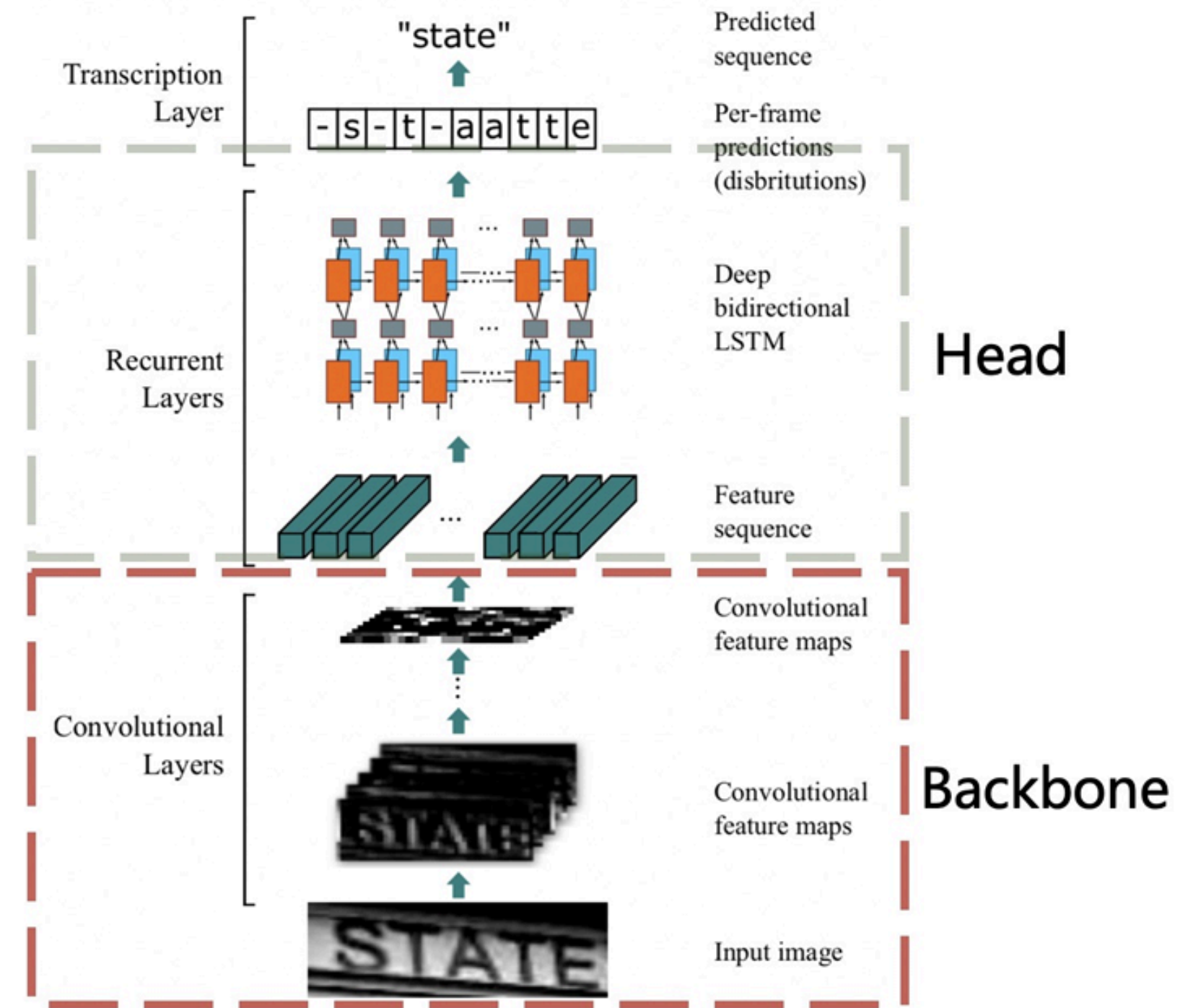


Training and Models usage

OCR Model - PaddleOCR

PaddleOCR's architecture has three main components:

- **Text Detection:** Uses models like DBNet, EAST, or CTPN to locate text regions.
- **Text Recognition:** Converts text images into character sequences using models like CRNN or SVTR.
- **Text Direction Classification:** Identifies whether text is horizontal or vertical. (Used in Chinese or Japanese documents)



Model Architecture

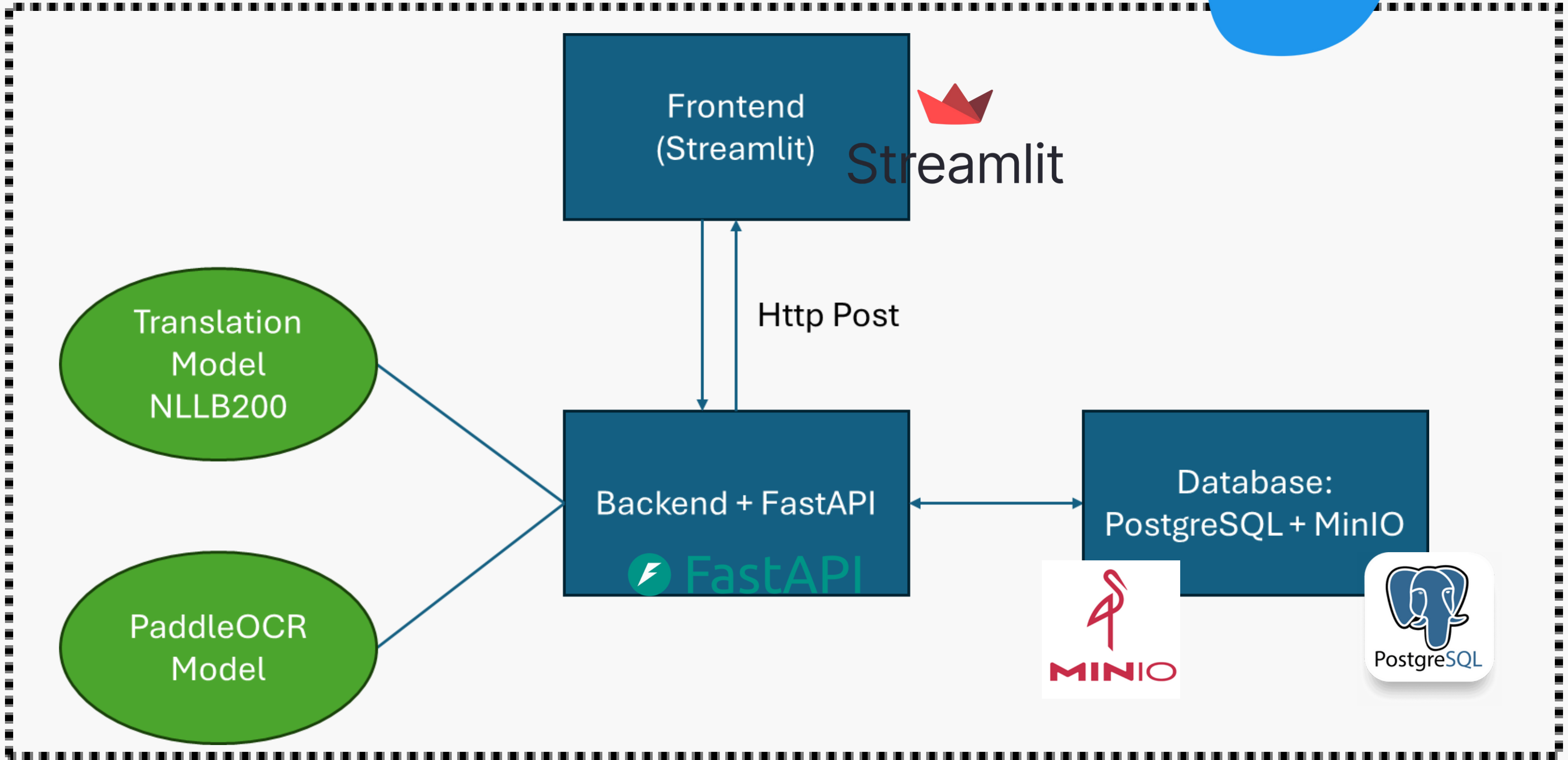
Training and Models usage

OCR Model - PaddleOCR

- We **fine-tuned PaddleOCR** on 8,000 images extracted from PDF documents to adapt it to our specific data. After training, we saved the model checkpoints and selected the best-performing weights for inference.
- However, on the test set, the fine-tuned model underperformed compared to the original pretrained version, **achieving a mAP score of 0.36 versus 0.47**.
- This suggests that the pretrained model generalizes better to our test data, despite the domain-specific training.

Building Application - Pipeline

Docker



Building Application

Backend - FastAPI

- We use **FastAPI** for our backend to handle requests and endpoints.
- Our backend uses the **PaddleOCR library** to extract text from images in multiple languages and a **fine-tuned nllb-200-600M translation model** to translate extracted or input text into Vietnamese
- After translating, the uploaded and translated images will be stored in **MinIO** while the ocr_data and translation will be stored in **Postgresql**.



Building Application

Backend - FastAPI

Our Endpoints are:

- **/set-lang/**: Sets the OCR language code.
- **/process-image/**: Performs OCR on uploaded image, translates text, and stores metadata in a database.
- **/upload-translated-image/**: Uploads a post-processed image to MinIO.
- **/translate-text/**: Translates a raw text string and stores the result.



Building Application

Frontend - Streamlit

- Language Selection: Users **choose a language from a dropdown** (based on langs.csv), setting PaddleOCR and NLLB codes, sent **via a /set-lang/ POST request**.
- File Upload: **Supports PNG, JPG, JPEG, PDF, and DOCX files**, with automatic file type detection and handling.



Streamlit

Building Application

Frontend - Streamlit

- OCR + Translation Flow: Sends image to **/process-image/**, overlays translated text using PIL, **displays translated image to screen** and **uploads the result to MinIO** via **/upload-translated-image/**.



Streamlit

Building Application

Database - PostgreSQL

- PostgreSQL - Structured Data: PostgreSQL is used to store ocr_data, translation data, and bounding boxes points detected.
- In our implementation, we host our **PostgreSQL in Docker.**



Building Application

Database - PostgreSQL

Docker Configuration:

- Image: postgres:15
- Environment Variables:
- Volume:
./postgres_data:/var/lib/postgresql/data
- Exposed Port: 5432
- Healthcheck:
- Command: pg_isready -U appuser -d ocrtranslate
- Interval: 5 seconds
- Retries: 5



Building Application

Database - MinIO

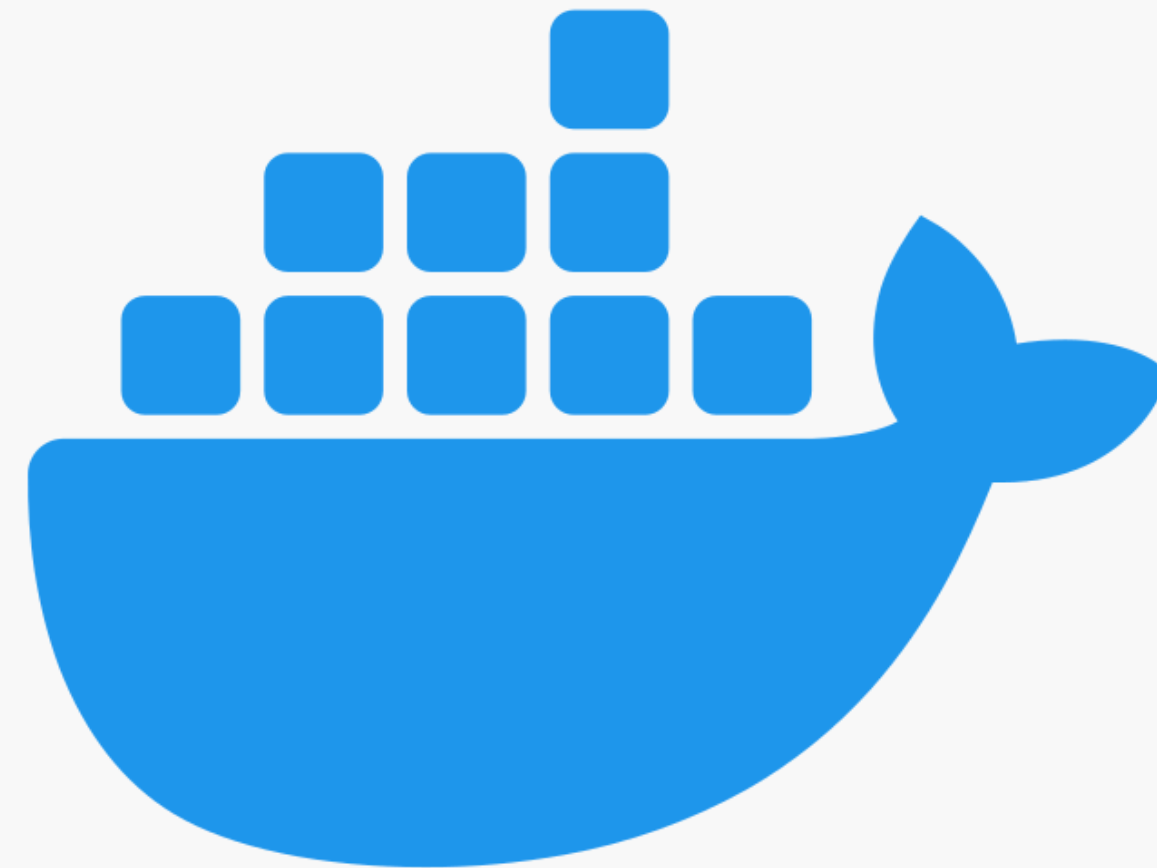
- MinIO - Unstructured Data: We use MinIO to store uploaded images and translated images.
- In our implementation, we host our **MinIO in Docker**.



Building Application

Docker

- We build and run our whole application using Docker
- When running Docker, there will be 4 services running at the time namely: **backend, frontend, postgresql and minio.**



Building Application

Docker

- Here are our docker images when running server

Images [Give feedback](#)

Local

Hub

4.08 GB / 15.06 GB in use

4 images

Last refresh: 0 seconds ago

Q Search

☐	Name	Tag		Created	Size	Actions	Image ID
☐	gdgoc-translation-app-backend	latest	●	17 hours ago	3.72 GB		22f9da41f7ee
☐	gdgoc-translation-app-ui	latest	●	3 days ago	843.62 MB		7d4608d78aa9
☐	postgres	15	●	16 days ago	608.44 MB		d393e37c6df9
☐	minio/minio	latest	●	1 month ago	250.04 MB		a1ea29fa2835

Building Application

Running application

To build and run our application follow the steps:

- Install Docker on your computer.
- In the root project folder, create a .env file with the format (given in our Github repo).
- Run docker-compose up --build in the root folder. Wait until the backend (uvicorn) is ready.
- Access the web at localhost:8501, MinIO at localhost:9001, and view OCR and translation data in PostgreSQL.
- For detailed instructions, see the README on our GitHub.